

MARKET MICROSTRUCTURE • RECONSTRUCTION FROM OHLCV

The Reconstructed Footprint

Volume tells you **how much** traded. A footprint tells you **where**. This is how you rebuild the second one from the first — with geometry, a truncated normal, and an arithmetic self-check that admits when it is lying.

Source • Pine v6, ata_sabanci Port • Python 3.12, stdlib only Residual • 0.02 PPM

The question

A candle cannot answer the only question that matters

A five-minute bar closes at 104.25, having traded between 104.10 and 104.35 on 1,894 lots. Now answer this: *did the buyers do their work at 104.12 and get paid, or did they chase 104.34 and get filled by someone unloading?*

Those are opposite situations. They produce the same candle.

A real footprint chart answers it by counting every print at every price and tagging each one by whether it lifted the ask or hit the bid. That needs a per-tick feed with aggressor flags — exchange-grade data most retail plans do not carry, and none of the free ones do.

This indicator's contribution is that it **reconstructs** the footprint from ordinary OHLCV, using a geometric model of how price traversed its own range. Then — and this is the part worth stealing regardless of whether you ever use the tool — it **audits its own arithmetic in parts per million** and tells you when the picture has stopped being faithful.

Six stages. Each one is a small, checkable idea. Here is what they add up to on a chart — every number below is real output from the port, not a mockup:

FIGURE 1 • THE ASSEMBLED VIEW

23-bar period • 6-column window • geometric engine



THE FOOTPRINT TABLE

the six most recent candles that traded inside the frame, at one-tick resolution

PRICE	0	-1	-2	-3	-4	-5
104.70	Δ	Δ	Δ	Δ	Δ	Δ
104.65	Δ	Δ	Δ	Δ	Δ	Δ
104.60	Δ	Δ	Δ	Δ	Δ	Δ
104.55	Δ	Δ	Δ	Δ	Δ	Δ
104.50	Δ	Δ	Δ	Δ	Δ	-68 +11
104.45	Δ	Δ	Δ	Δ	Δ	Δ-X -152 +36
104.40	Δ	Δ	Δ	Δ	-105 +3	Δ-X -160 +62
104.35	-114 +26	Δ	-45 +26	Δ	Δ-X -268 +13	-152 +96
104.30	Δ-X -249 +113	-86 +49	Δ-X -92 +69	Δ	Δ-X -330 +27	-129 +134
Δ104.25	-209 +227	Δ-X -161 +216	-92 +95	Δ	Δ-X -354 +51	-98 +167
104.20	Δ-X -124 +321	Δ-X -94 +369	-88 +124	-102 +3	Δ-X -330 +83	-66 +186
P 104.15	Δ-X -52 +321	Δ-X -32 +369	-80 +156	Δ-X -261 +112	Δ-X -268 +117	Δ-X -40 +186
104.10	-10 +128	-5 +130	Δ-X -70 +188	Δ-X -321 +26	-189 +144	Δ-X -22 +167
104.05	Δ	Δ	Δ-X -58 +215	Δ-X -344 +49	Δ-X -117 +154	-6 +72
104.00	Δ	Δ	Δ-X -46 +236	Δ-X -321 +80	-37 +75	Δ
103.95	Δ	Δ	Δ-X -35 +247	Δ-X -261 +114	Δ	Δ
103.90	Δ	Δ	Δ-X -26 +247	-184 +140	Δ	Δ
103.85	Δ	Δ	Δ-X -18 +236	Δ-X -113 +150	Δ	Δ
103.80	Δ	Δ	-12 +215	-36 +72	Δ	Δ

HOW TO READ IT

- sell +buy every cell is one price of one candle
- Δ-X Δ-X imbalance Δ-X buy vs the sell one tick below
- tint stronger = busier, scaled to the busiest cell
- Δ this candle never traded at this price
- P window point of control
- Δ-X row where price is right now Δ-X the ladder slides
- 0 -1 -2 bars back; headers jump when a candle is skipped

The table covers 103.80Δ104.70 Δ a 19-tick band around price. The profile above covers the whole 103.65Δ105.45 period range. Two resolutions of the same order flow.

Four things are on screen at once. **The candles** are the 23-bar period the profile covers. **The two bells** open rightward from a shared baseline one bar past the last candle – blue fed only by buy volume, red only by sell, on one horizontal scale so their relative size means something. **The lines** are the POC and the value-area edges, drawn back across the bars they were computed from, with the dashed grey pair capping the period's highest high and lowest low. **The table** is the footprint at one-tick resolution. The two amber labels are the tool talking about itself – the balance verdict at the ceiling, the arithmetic self-check at the floor.

Note the two resolutions: the profile spans the full 103.65Δ105.45 period range, while the table zooms into a 19-tick band around the current price. Same order flow, two magnifications. On a live chart the table is a corner *overlay* sitting on top of the candles; it is drawn below here so nothing covers anything.

THE ALTIMETER

One detail that confuses people on first contact: the table's price ladder does not stay still. It is anchored to the row of the *current price* and slides as price moves — like an aircraft altimeter, where the needle holds position and the scale rotates behind it. Price stays centred; the prices around it change. That is why the table always shows you the levels that matter right now instead of the levels that mattered when you opened the chart, and why a candle that never traded in the visible band silently loses its column.

Where buy and sell volume come from

Everything downstream reads exactly two numbers per bar: buy volume and sell volume. The engine decides how those two are produced, and *nothing else in the pipeline changes*. That separation is the reason the same code serves a free chart and a Premium per-tick feed.

FIGURE 2 • THE THREE ENGINES			<i>fidelity rises left to right; availability falls</i>
ENGINE	HOW IT SPLITS THE BAR	WHAT IT CANNOT SEE	NEEDS
Geometric	buy = $V\hat{A} \cdot (C\hat{A}^{\wedge}L) / (H\hat{A}^{\wedge}L)$ sell = $V\hat{A} \cdot (H\hat{A}^{\wedge}C) / (H\hat{A}^{\wedge}L)$	Any path that ran up and got sold back into. It reads the close, not the journey.	Nothing. Any OHLCV, any plan.
Intrabar	Sums directional volume from lower-timeframe candles inside the bar.	What happened <i>inside</i> each LTF candle. Resolution is bounded by your LTF choice.	Intrabar history “limited by plan and by how far back you look.
Footprint	Real per-price rows with aggressor-tagged buy and sell volume.	Nothing material. This is the measurement the other two estimate.	A per-tick feed. Premium/Ultimate on TradingView.
The Geometric split reads the candle as a story: a bar that closed on its high was lifted all session, so most of its volume was buyer-initiated. It is an <i>estimator</i> . The other two engines exist because sometimes an estimate is not good enough.			

THE PURITY RULE

A bar the engine cannot fill returns `None` “never `0.0`”. Missing data is missing. A bar with no reading is *excluded* from the window, not counted as a bar where nothing traded. Get this wrong and every gap in your history quietly votes for “balanced, no volume here,” which is the most dangerous wrong answer this tool can give.

02 Row split

Two bells, and the half-journey each one describes

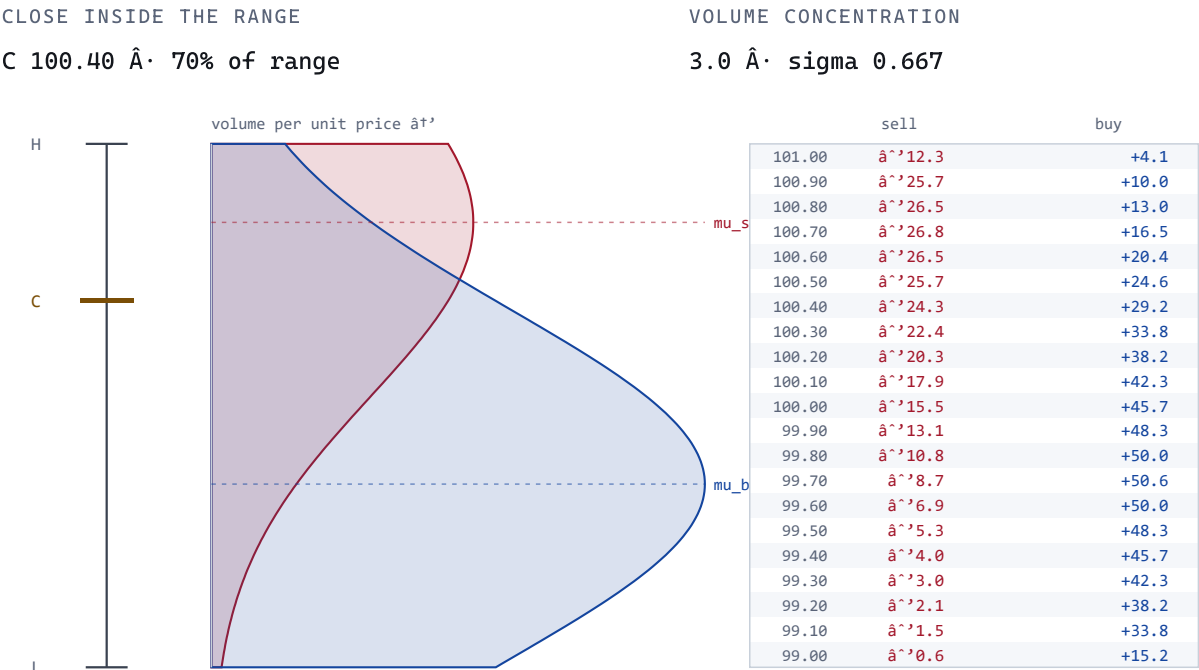
We now know a bar traded 700 buy and 300 sell. We still don't know *at which prices*. The model answers with two truncated normal distributions, and the centres are the whole idea:

```
// buyers worked the leg from the low up to the close mu_b = L + (C - L) / 2 // sellers
worked the leg from the close up to the high mu_s = C + (H - C) / 2 // one shared
width, set by the Concentration input sigma = (H - L) / concentration
```

Read it as a story of the bar. Price left the low, buyers pushed it to the close – so their volume centres *midway through that leg*. From the close price reached up to the high and came back – sellers did that work, so their volume centres midway through *that* leg. Two half-journeys, two centres.

Move the close and watch what happens. This is the single most useful thing to internalise about the tool:

FIGURE 3 • INTERACTIVE • THE BELL SPLITTER H 101.00 • L 99.00 • V 1,000 • rows shown at 0.10



BUY SHARE	BUY VOLUME	SELL VOLUME	MU BUY	MU SELL
0.700	700.0	300.0	99.700	100.700
ROWS + OFF-FRAME				
1000.000				

Drag the close to the high: the two bells collapse together near the top and the sell side thins to nothing. Drag it to the low: the reverse. Raise Concentration and both bells tighten toward their own centres – it is a width control, not a shape control. The last readout is the conservation check: rows plus off-frame mass always re-add to 1,000.

The lattice, and why rows are never merged

Row `k` is the absolute tick index `round(price / tick)`, and it owns the price band half a tick either side of its own level. A row's value is the *exact* truncated-normal mass of that band, clipped to the bar range:

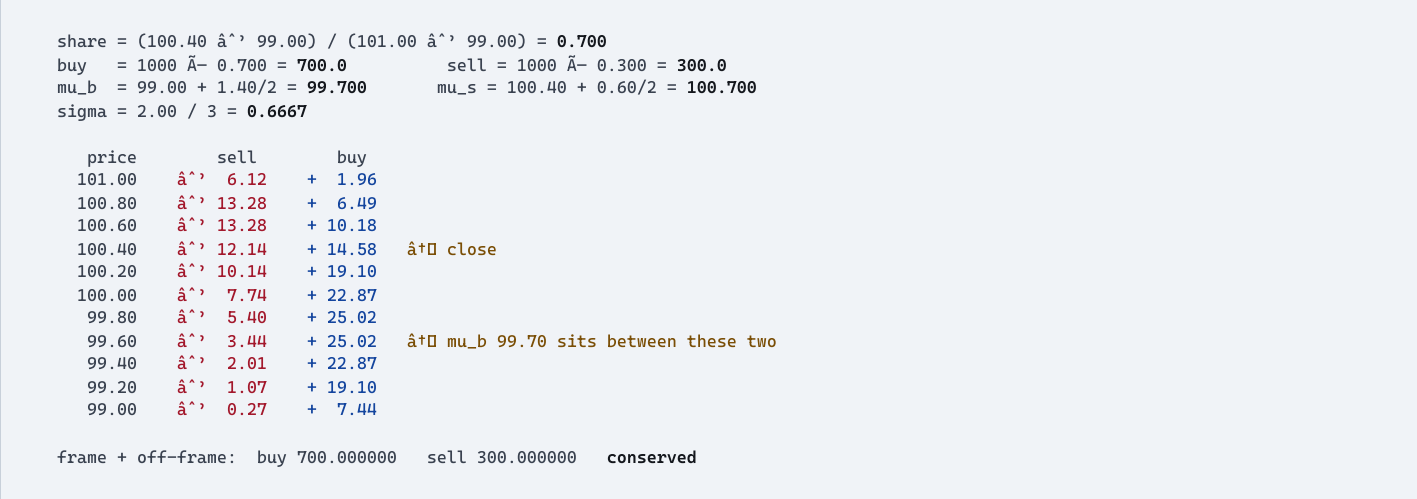
```
row_k = w * [ I((top_k - mu)/f) - I((bot_k - mu)/f) ] / Z Z = I((H - mu)/f) - I((L - mu)/f) // truncation mass
```

Three consequences worth naming:

- **Truncation is not cosmetic.** A bar cannot trade at a price it never printed, so the bell is cut off at H and L and re-normalised by its own surviving mass `Z`. Skip the `Z` and your two sides no longer add to the bar's volume.
- **Nothing is rescaled to fit the screen.** Volume that falls outside the visible frame is carried in a separate off-frame sum rather than dropped or squeezed into the edge rows. Per side, *frame rows + off-frame = bar volume*. That invariant is what stage o6 audits.
- **Flat bars are point atoms, not bells.** A zero-range bar has $\tilde{f} = 0$ and no distribution. It enters the model as a point mass at its own price and is carried through every later calculation separately. This is the case that breaks naive implementations “ on a limit-locked commodity, an illiquid option strike, or a halted stock, you get a divide-by-zero or a silently vanished bar.

FIGURE 4 · ONE BAR, WORKED

$H\ 101\ \hat{A}\ L\ 99\ \hat{A}\ C\ 100.40\ \hat{A}\ V\ 1,000\ \hat{A}\ concentration\ 3$



Only every fourth row is printed. The buy mass peaks around 99.60–99.80 “ the middle of the low-to-close leg “ while sell mass peaks around 100.60–100.80, the middle of the close-to-high leg. The centroid of buy volume always sits below the centroid of sell volume within a single bar; that is a testable claim, and the port tests it.

03 Imbalance

Why the comparison is diagonal

Now that each bar is a column of price rows, we can ask which levels were one-sided. The obvious test "compare buy and sell *at the same price*" is wrong, and understanding why is the fastest way to understand order flow generally.

A limit order book quotes a bid and an ask one tick apart. An aggressive buyer does not trade "at the price" they **lift the ask**. An aggressive seller **hits the bid**, one tick below. So buy volume at level k and sell volume at level k are prints that *never met each other*. The trades that actually competed are buy at k against sell at $k+1$.

```
buy_imbalance[k] = buy[k] > sell[k+1] - ratio sell_imbalance[k] = sell[k] >
buy[k+1] - ratio // ratio 3.0 (300%) is the classical figure
```

FIGURE 5 • THE DIAGONAL PAIRING

threshold 300% • cells read as sell + buy



$412 > 24 \rightarrow 3.00 \rightarrow$ BUY IMBALANCE

the aggressive buy lifted the ask;
the bid it competed with sat one tick under

412 vs 18 "same level, never met
this is the comparison that looks obvious
and tells you nothing about who won

Untraded rows stay silent "a missing value is never read as zero opposing volume."

If you treated an untraded row as zero opposing volume, every edge of every bar would be "infinitely imbalanced" and the signal would be pure noise. The `None`-is-not-zero rule from stage 01 is what keeps this readable.

WHAT TO DO WITH IT

A stack of buy imbalances on the way up is aggression being *absorbed* or aggression *working* – you cannot tell which from the imbalance alone, only from what price does next. The tool marks only the **first** imbalanced level above and below the POC, because in a trending profile half the ladder is imbalanced and a chart with thirty dotted lines says nothing. The first level away from the POC on each side is where the auction stopped being two-sided. That is the edge worth watching.

04 Profile

A period of bars, collapsed into two curves

The table reads bar by bar. The profile reads the *period*: every bar's two bells are summed into one mixture per side, and the result is drawn sideways off the right edge of the chart. Where the two curves overlap, both sides were doing business at the same prices.

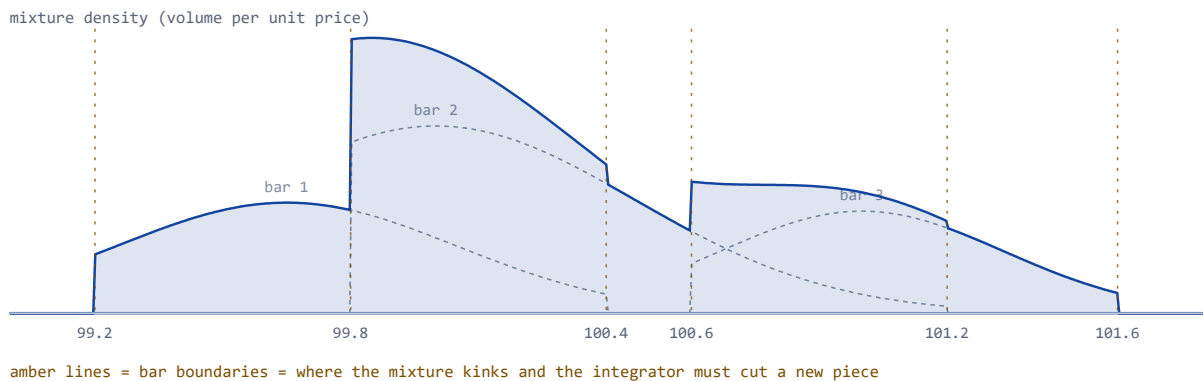
Each bar contributes one component per side, with an intensity coefficient that carries its volume:

```
coef = w / (f - Z) density(p) = ∫ over components whose range contains p: coef ·
f + ((p - μ) / f) // unit: volume per unit price — a density, not a probability
```

Divide that by the side's total volume and it becomes the mixture's probability density, which is what stage 06 needs. Keep it undivided and it is directly comparable with a table row — which is why the chart POC label can report the same *kind* of number as the dashboard POC.

FIGURE 6 — THE MIXTURE, FROM THREE BARS

each bar's bell is truncated to its own range



The mixture is not smooth. It kinks at every bar boundary, because that is where a component switches on or off. Those kinks are exactly where the integrator has to cut its pieces — see stage 06.

05 POC & Value Area

The two levels every profile trader already knows

These are classical Market Profile statistics, and the indicator's contribution is not inventing them but computing them *twice from one implementation* – once over the table's window of bars, once over the profile's period. If the two halves of a tool disagreed about what a POC is, they would be telling different stories.

Point of Control

`argmax(buy + sell)`. The price where the market did the most business. In a rotating market it is a magnet – price keeps returning to where the inventory sits. In a trending one it is a reference: a break away from the POC that *holds* is one of the cleaner signs the auction has genuinely moved rather than merely stretched.

Value Area

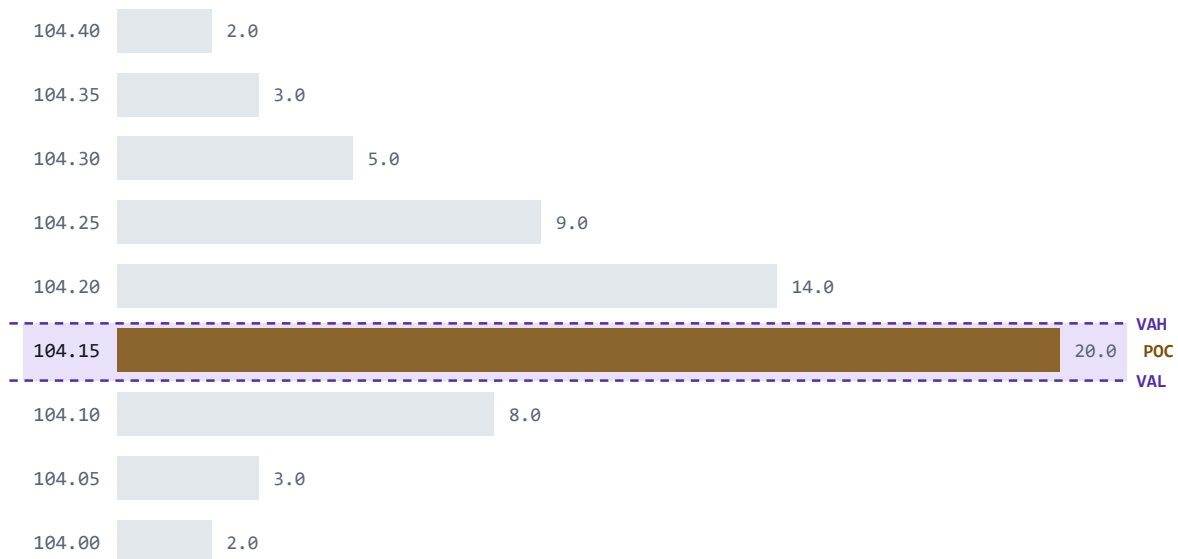
Start at the POC. Look at the neighbour above and the neighbour below, take whichever holds more volume, add it to the band. Repeat until the band holds 70% of the period's volume.

Seventy is not arbitrary: it is roughly one standard deviation of a normal distribution, and that is where the convention comes from. Inside the band the market *accepted* price and kept building business. Past the edges, trade thinned out fast.

FIGURE 7 • INTERACTIVE • VALUE AREA EXPANSION

press Step to add the heavier neighbour

inside band: 20.0 of 66.0 • target 46.2 • 30.3%



Note the expansion is stepwise “one row at a time” not the two-rows-at-a-time variant some platforms use. Neither is more correct; what matters is that the tool commits to one rule and applies it in both domains. The loop is boundary-guarded: an exhausted side is never stepped again, so it terminates even when the target can never be reached.

TRADING THE BAND

Two behaviours are worth watching. Price re-entering the value area from outside tends to *travel across it* to the far edge “the classical rotation trade. Price holding beyond an edge, refusing to come back, usually means the auction has moved on to do business elsewhere, and the old value area becomes context rather than support. The failure mode is treating the edges as fixed lines: they are the boundary of a *volume* statistic that updates every bar.

06 OVL • Tilt • RES

Three numbers, and only two of them are about the market

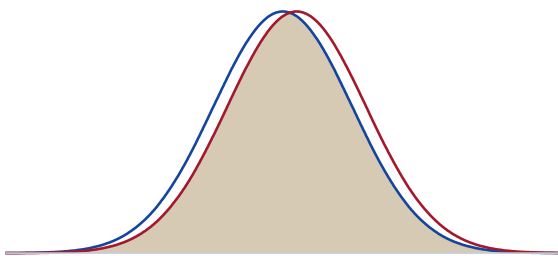
OVL • the overlapping coefficient

Normalise both sides to probability densities and integrate their pointwise minimum. The result runs 0 to 1 and answers a question no single-sided profile can: *did buyers and sellers work the same prices, or did each hold its own territory?*

FIGURE 8 • TWO PROFILES, SAME VOLUME, OPPOSITE MEANING

shaded region is the OVL integral

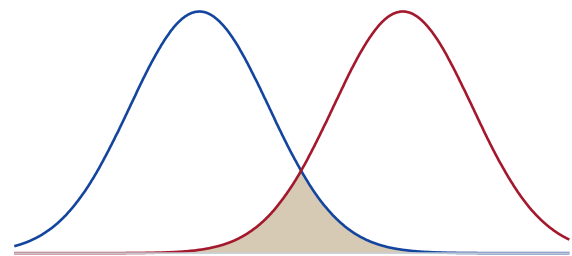
Balanced



OVL 0.92 • both sides worked the same prices

blue = buy density • red = sell density

Off balance



OVL 0.14 • each side held its own territory

blue = buy density • red = sell density

Left: OVL near 1. Both sides worked the same levels • rotation, and the profile edges tend to hold. Right: OVL well below the 0.75 boundary. Each side kept its own price territory, which is what a directional market physically looks like: one side pushing, the other answering somewhere else entirely.

Tilt • and the dead zone

$\text{tilt} = 100 \cdot (B - S) / (B + S)$. Delta expressed as a share of volume, so it is comparable across symbols and sessions in a way raw delta is not.

The design decision worth copying: when OVL says off-balance, the label only *names* a side if tilt clears a dead zone • 5 percentage points by default. Below that it says **OFF BALANCE** and refuses to pick. A two-point edge dressed up as a direction is a coin flip with a confident label on it, and that is worse than no label.

RES • the self-check nobody else ships

The area under each curve, plus the flat-bar atoms, has to add back to the volume that side actually traded. RES is how far apart the two came, in parts per million:

```
resid = | (â« density dp + atoms) / total_volume â' 1 | Ã- 1,000,000 // under tolerance
â' EXACT over â' DRIFT
```

This is a **health reading of the tool, not a signal about the market**. EXACT means the drawing you are looking at is a faithful picture of the volume behind it. DRIFT means the shape has left its tolerance and should not be traded until it settles.

It is also the reason the integration is done carefully rather than on a uniform grid. Composite Simpson, on pieces cut at every bar boundary â€ where the mixture kinks â€ and, for the overlap term, cut again at every crossing of the two densities, located by bisection to machine tolerance. Each piece's active component set is fixed by its midpoint, so every integrand Simpson ever sees is smooth. Step size is bounded by the narrowest active sigma.

The payoff is measurable: the reference run comes out at **0.02 PPM** against a 1 PPM tolerance. A naive uniform grid over the same mixture lands in the percent range â€ a curve that looks perfectly plausible while having quietly lost three percent of its volume. You would never notice. That is precisely the point of shipping the check.

FIGURE 9 · TWO NORMAL CDFS, ON PURPOSE

the original uses both, in different places

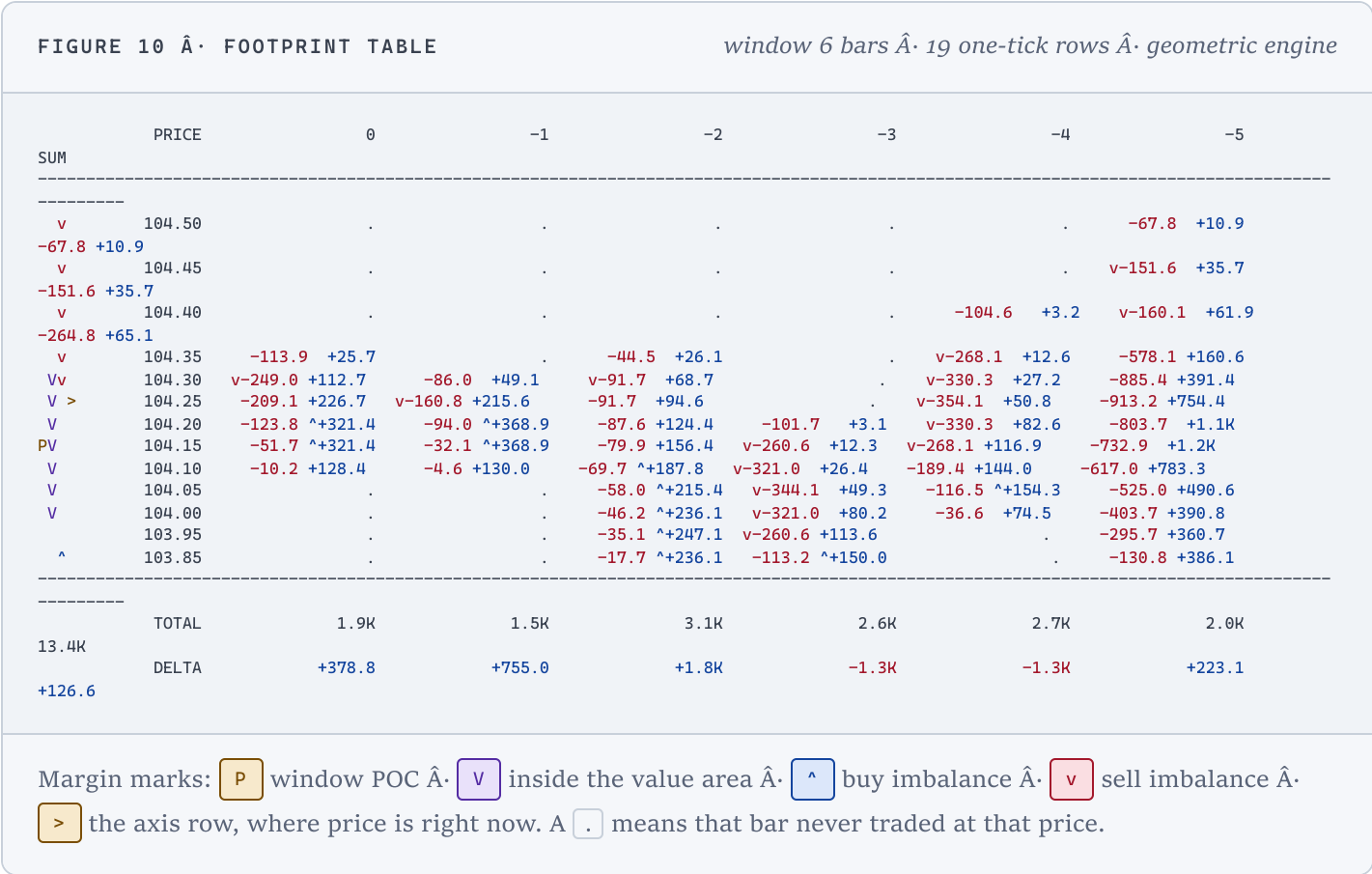
FUNCTION	ALGORITHM	ABSOLUTE ERROR	USED FOR
<code>norm_cdf_fast</code>	Abramowitz & Stegun 26.2.17	$< 7.5 \times 10^{-8}$	Per-bar table rows — thousands of calls per redraw, and eight digits is far beyond what a volume cell shows.
<code>norm_cdf_precise</code>	Cody rational Chebyshev <code>erfc</code>	$< 2 \times 10^{-16}$	The profile — because it self-reports in PPM, and a 7.5×10^{-8} CDF would trigger DRIFT on its own.

This is a genuinely good piece of engineering judgement, and the port preserves it exactly: the cheap CDF where precision is invisible, the expensive one where the tool audits itself. Python gets the second one free from `math.erfc`; Pine had to hand-roll Cody's coefficients.

In practice

Reading the thing

Real output from the port, six columns of a synthetic session. Cells read ^sell +buy. Column headers are bars back from the anchor “ and they *jump*, because candles that never traded inside the frame take no column at all. Every column you see is a candle that genuinely left volume at these prices.



What this particular window says: the POC sits at 104.15 with a value area of 104.00–104.30 “ price is at 104.25, inside value and near its upper edge. Window delta is +126.6 on 13.4K of volume, which is nothing: under one percent. Columns 0, ^1 and ^2 were strongly bought; ^3 and ^4 were strongly sold. That is not a trend, it is a fight.

And the profile agrees “ OVL 82.66%, above the 0.75 balance boundary, tilt ^1.35pp, verdict BALANCED. Two independent statistics over two different windows telling the same story is worth more than either alone.

The code

What the Python port is

1,876 lines of Pine v6 â€” a pure-standard-library Python package. Every formula above is ported faithfully, including the two-CDF decision and the `None`-is-not-zero rule. 37 tests pin the invariants rather than the numbers, so a regression shows up as a broken promise.

FIGURE 11 • MODULE MAP

the layout follows the original's own sections

```
volume_footprint/
mathkit.py    the two normal CDFs, and why there are two
bars.py       Bar / BarSeries, the price lattice
engines.py    Geometric / Intrabar / Footprint volume splits
rows.py       one bar â€” one-tick rows, plus diagonal imbalance
profile.py    the truncated-normal mixture, OVL, RES self-check
metrics.py    POC, value area, balance tilt
indicator.py   window search, and the assembled result
render.py     terminal table, dashboard, ASCII + matplotlib profile
demo.py       runnable, synthetic or CSV
tests/        37 tests • 0.2s • no network
```

FIGURE 12 • USAGE

the one thing the port does that Pine cannot

```
from volume_footprint import BarSeries, Settings, VolumeEngine
from volume_footprint import apply_engine, compute, format_table

series = BarSeries.from_ohlc(rows, mintick=0.05, symbol="NIFTY")
series = apply_engine(series, VolumeEngine.GEOMETRIC)

res = compute(series, Settings(windowBars=6, view_ticks=9, profile_period=23))

res.window_poc_price      # POC over the table's window
res.chart_poc_price       # POC over the profile's period
res.chart_vah_price, res.chart_val_price
res.profile.overlap       # OVL, 0-100
res.profile.tilt          # percentage points, + = buy side
res.profile.residual_ppm  # RES self-check
res.balance_verdict()     # "BALANCED" / "OFF BALANCE TO BUY" / ...

compute(series, end_offset=250) # â€” evaluate 250 bars ago
```

Pine can only run this block on `barstate.islast`, so its readings cannot be backtested. `end_offset` walks history, which turns OVL, tilt and the POC levels into series you can actually study. That is the main practical reason to port it at all.

What the port deliberately leaves out

- **No Footprint data source.** The row reader is implemented and tested, but you must supply real per-tick rows. Inventing an offline substitute would defeat the purpose.

- **No chart objects.** Label merge priorities, colour ladders and table anchoring are TradingView presentation concerns. The numbers are all here.
- **No alerts.** The control levels are exposed as plain values “” wire them into whatever you already run.

Tutor's notes

How to actually use this, and where it will fool you

The reading order that works

1. **RES first.** If it says DRIFT, stop. Everything below is unreliable and you are about to trade a rendering artifact.
2. **OVL second.** It tells you which *kind* of market you are in, and therefore which of the remaining numbers mean anything. Balanced $\hat{+}$ the profile edges are tradable and mean-reversion has an edge. Off balance $\hat{+}$ the edges are targets, not walls, and fading them is fighting the auction.
3. **POC and value area third.** Location relative to price is the setup. Price inside value near an edge is a different trade from price outside value trying to get back in.
4. **Delta and imbalance last.** These are confirmation, not triggers. Delta is the noisiest number on the screen and the most likely to be an artifact of the engine.

Parameters that actually change the answer

Concentration is the one to be careful with. It sets $\tilde{f} = \text{range} / \text{concentration}$, so it controls how tightly volume gathers at each bell's centre. High values produce sharp, confident-looking peaks that are *entirely a modelling choice* $\hat{+}$ you are not discovering concentration in the market, you are asserting it. The default of 3 puts roughly $\hat{\pm} 1.5\tilde{f}$ across the bar range, which is a reasonable statement of "we don't know much." If you raise it, you should be able to say why.

Profile period and **window bars** are your two timeframes: the period is the structure you are trading against, the window is what is happening right now. Keeping them related $\hat{+}$ a session period, a recent-bars window $\hat{+}$ is what makes the dashboard and the chart usable together.

Ticks above/below only affects what you can *see*. Volume outside the frame is carried, not lost, so the window POC still counts only what is in frame. A frame too tight will put the POC on a boundary row, which is a tell that you should widen it.

THE HONEST CAVEAT

The Geometric engine is a **model, not a measurement**. Its buy/sell split is a geometric assumption about how price traversed its range, and two completely different order-flow paths produce the same candle and therefore the same reconstruction. Treat POC, value area and OVL from this engine as *structure* where the volume sat not as verified aggressor flow.

Where a decision hinges on who was actually the aggressor, use the Intrabar engine at a meaningfully lower timeframe, or a genuine per-tick feed. And note what that costs: intrabar history is short, so the deeper you look back the more bars return no data at all correctly, by the purity rule, rather than being quietly filled in.

RES saying EXACT confirms the integration is faithful *to the model*. It says nothing whatsoever about whether the model matches what traded.

The three ideas worth stealing even if you never run this

- **Ship the residual.** Any indicator that redistributes a conserved quantity volume, open interest, gamma can check that the redistribution conserves it, and report the drift. Almost none do. The check costs nothing and catches the class of bug that produces beautiful, confident, wrong pictures.
- **Missing is not zero.** One boolean's worth of discipline, and it is the difference between "no data here" and "nothing traded here" which point in opposite directions.
- **Refuse to call it when you cannot.** The tilt dead zone is a tool declining to produce a signal it does not have. That is rarer and more valuable than any indicator that always has an opinion.

Source: [Volume Footprint: Measuring Classical Indicators by Math & Geometry](#) Pine v6 by [ata_sabanci](#), MPL-2.0. The Python port carries the same licence. All numbers on this page are real output from the port, not illustrations.